

Sorting Algorithms

Junior ծրագրավորողի հարցազրույցի ամբողջական ուղեցույց

Այս presentation-ը ընդգրկում է բոլոր հիմնական sorting algorithm-ները, որ junior interview-ում կարող են հարցնել՝ ամեն մեկը C++ կոդով, քայլ-առ-քայլ բացատրությամբ, time/space complexity-ով, stability-ով, և «երբ օգտագործել» խորհուրդով: Վերջում կա համեմատական աղյուսակ և ամենահաճախ հարցվող interview հարցերը:

0. Հիմնական հասկացություններ

Նախքան algorithm-ները, պետք է իմանալ 4 termin, որ interview-ում միշտ հարցնում են:

Time Complexity — քանի քայլ է պահանջում ալգորիթմը կախված input-ի չափից (n): Չափվում է Big-O-ով՝ $O(n^2)$, $O(n \log n)$ և այլն:

Space Complexity — որքան լրացուցիչ հիշողություն է պետք: $O(1)$ = in-place (հաստատուն հիշողություն), $O(n)$ = լրացուցիչ array:

Stability — stable sort-ը պահպանում է հավասար էլեմենտների սկզբնական հերթականությունը: Կարևոր է, երբ sort անում ես object-ներ ըստ մի քանի key-ի:

In-place — ալգորիթմը sort է անում հենց նույն array-ում, առանց մեծ լրացուցիչ հիշողության:

Best case -> ամենալավ դեպք (արդեն sorted input)

Average case -> միջին դեպք (պատահական input)

Worst case -> ամենավատ դեպք (հակառակ sorted input)

1. Bubble Sort

Գաղափարը

Ամենապարզ sort-ը: Բազմիցս անցնում է array-ով, հարևան էլեմենտները համեմատում և փոխում (swap), եթե սխալ հերթականությամբ են: Ամեն անցքից հետո ամենամեծ էլեմենտը «պղպջակի պես» բարձրանում է վերջ:

Քայլ առ քայլ ([5, 2, 8, 1])

Pass 1: [2,5,8,1] -> [2,5,8,1] -> [2,5,1,8] (8-ը գնաց վերջ)
Pass 2: [2,5,1,8] -> [2,1,5,8] (5-ը տեղը գտավ)
Pass 3: [1,2,5,8] (sorted)

Կոդ

```
void bubbleSort(vector<int>& a) {  
    int n = a.size();  
    for (int i = 0; i < n - 1; i++) {  
        bool swapped = false;  
        for (int j = 0; j < n - 1 - i; j++) {  
            if (a[j] > a[j + 1]) {  
                swap(a[j], a[j + 1]);  
                swapped = true;  
            }  
        }  
        if (!swapped) break; // արդեն sorted է -> կանգնիր  
    }  
}
```

Բնութագրեր

Time: Best $O(n)$ | Average $O(n^2)$ | Worst $O(n^2)$
Space: $O(1)$ (in-place)
Stable: Այո

`swapped` flag-ը կարևոր optimization է. եթե մի ամբողջ pass-ում ոչ մի swap չեղավ, array-ն արդեն sorted է -> Best case `O(n)` :

2. Selection Sort

Գաղափարը

Ամեն քայլում գտնում է մնացած (unsorted) մասի ամենափոքր էլեմենտը և դնում sorted մասի վերջում: Array-ը բաժանվում է sorted (ձախ) և unsorted (աջ) մասերի:

Քայլ առ քայլ ([5, 2, 8, 1])

```
Գտիր min(5,2,8,1)=1 -> swap 5,1 -> [1,2,8,5]
Գտիր min(2,8,5)=2   -> տեղում   -> [1,2,8,5]
Գտիր min(8,5)=5      -> swap 8,5 -> [1,2,5,8]
```

Կոդ

```
void selectionSort(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIdx])
                minIdx = j;
        swap(a[i], a[minIdx]);
    }
}
```

Բնութագրեր

Time: Best $O(n^2)$ | Average $O(n^2)$ | Worst $O(n^2)$
Space: $O(1)$ (in-place)
Stable: Ոչ (swap-ը կարող է քանդել հերթականությունը)

Առանձնահատկություն. միշտ $O(n^2)$, նույնիսկ sorted input-ի դեպքում: Բայց անում է ամենաքիչ swap-երը ($O(n)$), ինչը լավ է, երբ swap-ը թանկ է:

3. Insertion Sort

Գաղափարը

Ինչպես խաղաքարտերը ձեռքում դասավորելը: Ամեն էլեմենտ վերցնում է և տեղադրում sorted մասում իր ճիշտ տեղը՝ մեծ էլեմենտները մեկ քայլ աջ տեղաշարժելով:

Քայլ առ քայլ ([5, 2, 8, 1])

```
[5 | 2,8,1]  վերցրու 2 -> [2,5 | 8,1]
[2,5 | 8,1]  վերցրու 8 -> [2,5,8 | 1]
[2,5,8 | 1]  վերցրու 1 -> [1,2,5,8]
```

Կոդ

```
void insertionSort(vector<int>& a) {
    int n = a.size();
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];    // տեղաշարժ աջ
            j--;
        }
        a[j + 1] = key;        // տեղադրիր ճիշտ տեղում
    }
}
```

Բնութագրեր

Time: Best $O(n)$ | Average $O(n^2)$ | Worst $O(n^2)$
Space: $O(1)$ (in-place)
Stable: Այո

Ամենալավն է փոքր կամ գրեթե-sorted array-ների համար (Best $O(n)$): Հենց սա է օգտագործում `std::sort` -ը փոքր subarray-ների համար:

4. Merge Sort

Գաղափարը

Divide and Conquer. array-ը կիսում է երկու մասի, ամեն մասը ռեկուրսիվ sort անում, ապա երկու sorted մասերը միավորում (merge) մեկ sorted array-ի:

Քայլ առ քայլ ([5, 2, 8, 1])

Divide: [5,2,8,1] -> [5,2] [8,1] -> [5][2] [8][1]

Merge: [5][2]->[2,5] [8][1]->[1,8]

Merge: [2,5]+[1,8] -> [1,2,5,8]

Կոդ

```
void merge(vector<int>& a, int l, int m, int r) {
    vector<int> left(a.begin() + l, a.begin() + m + 1);
    vector<int> right(a.begin() + m + 1, a.begin() + r + 1);
    int i = 0, j = 0, k = l;
    while (i < left.size() && j < right.size())
        a[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while (i < left.size()) a[k++] = left[i++];
    while (j < right.size()) a[k++] = right[j++];
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
```

Բնութագրեր

Time: Best $O(n \log n)$ | Average $O(n \log n)$ | Worst $O(n \log n)$
Space: $O(n)$ (լրացուցիչ array merge-ի համար)
Stable: Այո

Երաշխավորված $O(n \log n)$ ամեն դեպքում: Օգտագործվում է, երբ stability պետք է, կամ linked list-երի ու արտաքին (external) sort-ի համար:

5. Quick Sort

Գաղափարը

Divide and Conquer. ընտրում է pivot էլեմենտ, array-ը partition է անում` pivot-ից փոքրերը ձախ, մեծերը աջ, ապա ռեկուրսիվ sort անում երկու մասերը:

Քայլ առ քայլ ([5, 2, 8, 1] , pivot = վերջին)

```
pivot=1: partition -> [1 | 5,2,8]    (1-ից փոքր չկա)
pivot=8: partition -> [5,2 | 8]
pivot=2: partition -> [2 | 5]... -> [1,2,5,8]
```

Կոդ

```
int partition(vector<int>& a, int low, int high) {
    int pivot = a[high];           // pivot = վերջին էլեմենտ
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;                  // pivot-ի վերջնական դիրքը
}

void quickSort(vector<int>& a, int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

Բնութագրեր

Time: Best $O(n \log n)$ | Average $O(n \log n)$ | Worst $O(n^2)$
Space: $O(\log n)$ (recursion stack, in-place partition)
Stable: Ոչ

Worst case $O(n^2)$ -ը լինում է, երբ pivot-ը միշտ ամենափոքր/մեծ էլեմենտն է (օր. արդեն sorted array + վերջին pivot): Լուծում. random pivot կամ median-of-three: Գործնականում ամենաարագ sort-ն է cache-friendly լինելու պատճառով:

6. Heap Sort

Գաղափարը

Array-ից կառուցում է max-heap (ամենամեծը արմատում), ապա բազմիցս վերցնում է արմատը (max), դնում վերջ, և վերականգնում heap-ը:

Կոդ

```
void heapify(vector<int>& a, int n, int i) {
    int largest = i, l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}

void heapSort(vector<int>& a) {
    int n = a.size();
    for (int i = n / 2 - 1; i >= 0; i--) // build max-heap
        heapify(a, n, i);
    for (int i = n - 1; i > 0; i--) { // extract max
        swap(a[0], a[i]);
        heapify(a, i, 0);
    }
}
```

Բնութագրեր

Time: Best $O(n \log n)$ | Average $O(n \log n)$ | Worst $O(n \log n)$
Space: $O(1)$ (in-place)
Stable: Ոչ

Երաշխավորված $O(n \log n)$ + in-place (ի տարբերություն merge sort-ի): Բայց cache-unfriendly է, ուստի զործնականում quick sort-ից դանդաղ:

7. Counting Sort

Գաղափարը

Non-comparison sort: Հաշվում է ամեն արժեքի հանդիպումների քանակը count array-ում, ապա վերականգնում sorted array-ը: Աշխատում է միայն, երբ արժեքները սահմանափակ միջակայքում են (integers, փոքր range):

Կոդ

```
void countingSort(vector<int>& a) {
    if (a.empty()) return;
    int maxVal = *max_element(a.begin(), a.end());
    int minVal = *min_element(a.begin(), a.end());
    vector<int> count(maxVal - minVal + 1, 0);
    for (int x : a) count[x - minVal]++;
    int idx = 0;
    for (int i = 0; i < count.size(); i++)
        while (count[i]-- > 0)
            a[idx++] = i + minVal;
}
```

Բնութագրեր

Time: $O(n + k)$ (k = արժեքների range)
Space: $O(k)$
Stable: Այո (ճիշտ implementation-ում)

Կոտրում է $O(n \log n)$ սահմանը, քանի որ չի համեմատում էլեմենտները: Բայց range-ը (k) մեծ լինելիս անարդյունավետ է հիշողությամբ:

8. Radix Sort

Գաղափարը

Sort է անում թվերը թվանշան-առ-թվանշան (digit by digit)՝ ամենափոքր կարգից (units) դեպի ամենամեծը, ամեն քայլում օգտագործելով stable counting sort:

Կոդ

```
void countingByDigit(vector<int>& a, int exp) {
    int n = a.size();
    vector<int> output(n), count(10, 0);
    for (int i = 0; i < n; i++) count[(a[i] / exp) % 10]++;
    for (int i = 1; i < 10; i++) count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--) {    // հետից առաջ -> stable
        int d = (a[i] / exp) % 10;
        output[--count[d]] = a[i];
    }
    a = output;
}

void radixSort(vector<int>& a) {
    if (a.empty()) return;
    int maxVal = *max_element(a.begin(), a.end());
    for (int exp = 1; maxVal / exp > 0; exp *= 10)
        countingByDigit(a, exp);
}
```

Բնութագրեր

Time: $O(d * (n + k))$ (d = թվանշանների քանակ, $k = 10$)
Space: $O(n + k)$
Stable: Այո

Շատ արագ մեծ քանակությամբ integer-ների համար: Աշխատում է միայն ամբողջ թվերի (կամ fixed-format key-երի) վրա:

9. Shell Sort

Գաղափարը

Insertion sort-ի բարելավում: Համեմատում է հեռու (gap-ով բաժանված) էլեմենտները, gap-ը աստիճանաբար փոքրացնում մինչև 1: Սա թույլ է տալիս էլեմենտներին արագ շարժվել հեռու տեղեր:

Կոդ

```
void shellSort(vector<int>& a) {
    int n = a.size();
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = a[i];
            int j;
            for (j = i; j >= gap && a[j - gap] > temp; j -= gap)
                a[j] = a[j - gap];
            a[j] = temp;
        }
    }
}
```

Բնութագրեր

Time: Best $O(n \log n)$ | Average $\sim O(n^{1.25})$ | Worst $O(n^2)$
Space: $O(1)$ (in-place)
Stable: Ոչ

Insertion sort-ից էապես արագ, պարզ implement անել, in-place: Complexity-ն կախված է gap sequence-ից:

10. std::sort (STL)

Գործնականում ձեռքով sort չես գրում, օգտագործում ես STL-ը: Interview-ում կարևոր է իմանալ.

```
#include <algorithm>
vector<int> v = {5, 2, 8, 1};

sort(v.begin(), v.end());           // աճման կարգով
sort(v.begin(), v.end(), greater<int>()); // նվազման կարգով

// custom comparator
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b;
});

stable_sort(v.begin(), v.end()); // stable, պահպանում է հերթականությունը
partial_sort(v.begin(), v.begin() + 3, v.end()); // միայն առաջին 3-ը sort
```

`std::sort` -ը **Introsort** է, quick sort + heap sort (worst case-ից խուսափելու համար) + insertion sort (փոքր մասերի համար): Guaranteed $O(n \log n)$, unstable: `std::stable_sort` -ը merge-based է, stable, $O(n \log n)$:

11. Համեմատական աղյուսակ

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Այն
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ոչ
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Այն
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Այն
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Ոչ
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Ոչ
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Այն
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Այն
Shell	$O(n \log n)$	$\sim O(n^{1.25})$	$O(n^2)$	$O(1)$	Ոչ

12. Երբ որ sort-ը օգտագործել

Փոքր array (< 50)	-> Insertion sort
Գրեթե sorted	-> Insertion sort ($O(n)$)
Երաշխավորված $O(n \log n)$ պետք	-> Merge sort կամ Heap sort
Stability պետք է	-> Merge sort (կամ stable_sort)
Ընդհանուր, ամենաարագ միջին	-> Quick sort (std::sort)
Փոքր range integer-ներ	-> Counting sort
Մեծ integer-ներ, fixed digits	-> Radix sort
Հիշողությունը սահմանափակ	-> Heap sort (in-place, $O(1)$)
Իրական կոդ	-> std::sort / std::stable_sort

13. Ամենահաճախ Interview հարցերը

Ո՞ր sort-ն է ամենաարագը — Միջին դեպքում Quick sort (cache-friendly): Բայց worst case-ը $O(n^2)$ է; Երաշխավորված $O(n \log n)$ -ի համար Merge կամ Heap sort:

Ի՞նչ է stable sort-ը — Պահպանում է հավասար էլեմենտների սկզբնական հերթականությունը: Stable՝ Bubble, Insertion, Merge, Counting, Radix: Unstable՝ Selection, Quick, Heap, Shell:

Ինչո՞ւ չի կարելի $O(n \log n)$ -ից արագ (comparison sort) — Ցանկացած comparison-based sort-ի ստորին սահմանը $O(n \log n)$ է (decision tree-ի բարձրությունը): Counting/Radix-ը արագ են, քանի որ չեն համեմատում:

Quick sort-ի worst case-ը երբ է — Երբ pivot-ը միշտ ամենափոքր/մեծ էլեմենտն է (օր. sorted array + վերջին pivot): Լուծում. random pivot / median-of-three:

Merge vs Quick — Merge՝ երաշխավորված $O(n \log n)$, stable, բայց $O(n)$ հիշողություն: Quick՝ միջինում ավելի արագ, in-place, բայց worst $O(n^2)$, unstable:

In-place ի՞նչ է — Bubble, Selection, Insertion, Heap, Shell ($O(1)$): Merge-ը $O(n)$ է, ոչ in-place:

std::sort -ը ի՞նչ է ներսում — Introsort. quicksort + heapsort (fallback) + insertion sort (փոքր մասեր): $O(n \log n)$ guaranteed, unstable:

Cheat Sheet

$O(n^2)$ sorts: Bubble, Selection, Insertion (պարզ, փոքր data)
 $O(n \log n)$ sorts: Merge, Quick, Heap (արդյունավետ, մեծ data)
 $O(n)$ sorts: Counting, Radix, Bucket (non-comparison, հատուկ դեպքեր)

Stable: Bubble, Insertion, Merge, Counting, Radix

Unstable: Selection, Quick, Heap, Shell

In-place: Bubble, Selection, Insertion, Heap, Shell, Quick(log n stack)

Not: Merge ($O(n)$), Counting/Radix ($O(k)/O(n+k)$)

Guaranteed $O(n \log n)$: Merge, Heap

Fastest average: Quick (std::sort = Introsort)